# Integrated Development Environments (IDEs) for Object-Oriented Programming (OOP)

## Study Guide

## Introduction

Integrated Development Environments (IDEs) are an important if not essential tool that a programmer needs to design, test, and refine software in one cohesive workspace. IDEs streamline the coding process by combining editors, compilers, debuggers, and increasingly, AI-powered assistants into a single platform.

Mastering the use of IDEs in your Advancement Year Program (AYP) starting in Computer Science 3, is not just about knowing where things are inside an IDE, it is about using and applying its features to make you more productive, help you debug errors, and support the overall process of creative problem-solving in developing your code.

In this study guide, you will:

- Explore the basic features and components that make IDEs essential code and application development.
- Compare the capabilities of different IDEs to help you select which best suits specific programming tasks.
- Practice applying debugging and AI-powered tools to identify, correct, and improve code efficiently.

By the end of this study guide, not only will you be familiar with the mechanics of IDEs but you have prepared your working environment that you will be using throughout the school year in Computer Science 3. Your CS3 portfolio will be started in this study guide, and  will be used to document your work, and that if it is updated consistently, it will help you understand the course better. It can also be used as a reference when you transition to CS4 in the next school year.

The following module, which is a review of computational thinking skills, will be for revisiting old lessons in problem solving and algorithmic thinking. A diagnostic exam will be given at our next meeting, so prepare for this assessment so that your teacher will be able to address some gaps in your understanding.

## Module Learning Objectives

After working on this module, you should be able to:

1. Identify the basic features and components of an Integrated Development

Environment (IDE).

2. Compare the capabilities of at least two IDEs for a specific programming task.

3. Apply debugging and AI-powered tools in an IDE to correct errors and improve code.

**Study Schedule:**

| Day | Time | Activity | Key Concepts |
|---|---|---|---|
| 1<br>Face-To-Face<br>(50 minutes) | 10-15 minutes | Discussion | **Key Concept 1.1**<br><br>Basic Features and Components of an IDE and comparing different IDEs |
| | 30-40 minutes | Individual work | **Activity 1: Part A**<br><br>Exploring IDE Features<br>**(graded exercise)** |
| 2<br>Independent Learning Activity / Assignment | 30 minutes | Setup your personal computer to run python or setup your Github Account | **ILP Activity: Preparing your work environment to be found in the additional resources after Activity 1: Part B.**<br><br>Github account, installing Git, and VSCode with Python Extension.<br><br>**Note:** This is an optional activity in case students were not able to do this last school year and this is not graded but required for them to do for these students. |
| 3<br>Face-To-Face<br>(50 minutes) | 10-15 minutes | Short discussion on the activity Part A | |
| | 30-40 minutes | Demonstration and work along | **Activity 1: Part B**<br><br>Setting-up your CS3 Code portfolio<br><br>**(non-graded activity)** |
| Key<br>Take-Aways | | | **Optional Bonus Activity as an assignment to the student to be submitted in the manner dictated by your teacher.** |

The next section will give a summary of each learning resource and the requirements for each activity as outlined in the Study Schedule.

## Key Concepts and Learning Activities

### Learning resource 1: Integrated Development Environment

#### *1.1. Features and Components of an Integrated Development Environment*

An Integrated Development Environment (IDE) is a central hub where you can design, create, test, and refine your code. IDE integrates different components into one seamless system or workspace instead of using different applications.

**Task/s related to the learning resource**

Please read the following resources through the given link and watch the following short Youtube video that describes what an IDE is and the components that normally comprise the integrated development environment that application developers use.

1. What is an integrated development environment (IDE)? | Definition from TechTarget OR https://bit.ly/3OK3QiE included in this resource is a 1.38 minute of Youtube video explaining the importance of an IDE

As described in the resource, components that are normally found in an IDE are described below:

**The Core Components of an IDE**

- **Code Editor** – This workspace is where you write your code. It has syntax highlighters, suggests code, marks syntax errors, and flags potential mistakes as warnings.
- **Compiler/Interpreter** – Translates your human-readable code into machine instructions.
- **Application Preview/Run Window** - It is a space where you can actually see how your application will look and show results.
- **Debugger** – A tool that helps you trace errors step by step, and suggests solutions to help you create a code that is free from any logic or runtime errors.
- **Project Explorer** – This tool will help you create and manage projects, directories, and files.
- **Output Console** – Displays results, warnings, and error messages and can be used as a feedback loop for your program.
- **AI-powered Tools** – Modern IDEs include assistants that predict bugs, suggest improvements, or even generate snippets of code and now even generate an entire application based on your design.

  AI powered tools support vibe programming or vibe coding. For more information about vibe-coding you can refer to this **4 minute** youtube video What is Vibe Coding? (or type https://bit.ly/466qfNb) as extra information on this concept, and which is now a big thing in creating applications. **But keep in mind in PSHS all your graded exercises and your application projects should be created entirely by you. The use of AI should be guided by the policies set by your teacher in class.**

2. Several Youtube videos to compare different IDEs for specific programming languages (Choose only what programming language your class will be using for this SY):
   a. Python IDEs for 2026: Top 5 Best Python IDEs in 2026 #python or https://bit.ly/3OHu5Gu (duration 2:18 minutes)

   b. Java IDE Comparison in 2026: IntelliJ vs Eclipse vs VS Code: Power, Plugins or Performance? (2026) or https://bit.ly/4kCCAOY (duration 1:53 minutes)

   c. C++ IDE Comparison: What to Look for in a C++ IDE - YouTube or https://bit.ly/3OgkZR4 (Duration: 9:58minutes)

The second resource compares different IDEs per programming language. You can summarize that IDEs in the market are geared towards or work best with a specific programming language while others are more general-purpose, which could support different ones by just installing some libraries and extensions to support a programming language. Some are free to use, and some are proprietarial and you have to pay a subscription fee to be able to use their full features.

After reading the resources, Activity 1 is a simple exercise that will let you explore the features of a particular IDE.

**Activity 1:**

**Part A: IDE Features Hunt - tasks on creating and debugging codes.**

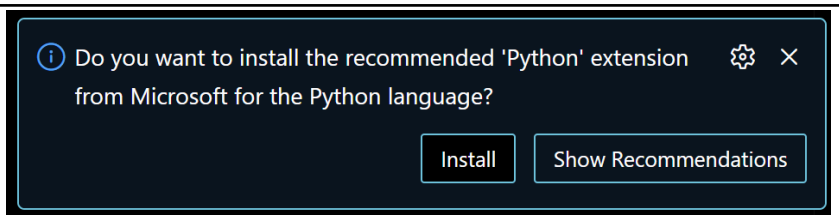
After reading and watching resources about Integrated Development Environments (IDEs), select the IDE that best fits your needs. Your choice should depend on the programming language you plan to use and the features that will make your work more efficient.

To simplify our discussion on Integrated Development Environments (IDEs), this study guide will use Visual Studio Code (VS Code) as the IDE for our laboratory activities. VS Code was selected as the IDE of choice for CS3 because it supports multiple programming languages, includes all the essential components discussed in the previous sections, and most importantly, it is free to use.

Beyond these advantages, VS Code integrates well with widely used version control tools: Git for local repository management and GitHub for cloud-based collaboration. This combination makes VS Code a powerful, flexible, and accessible environment for both beginners and advanced programmers.

**Instructions:**

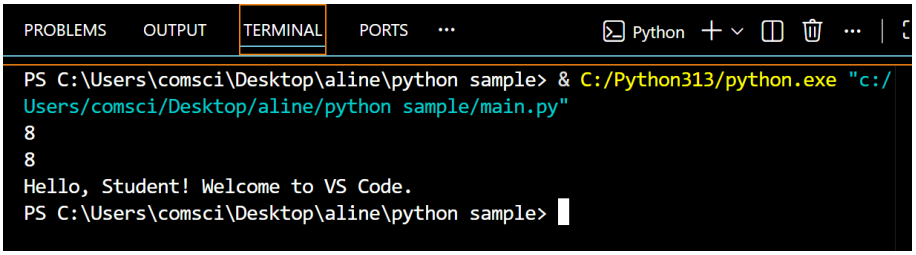
1. Please open your VSCode.
2. Open a new file and name it as main.py
3. VSCode will ask you to select a location where you can save the file. Create a folder in either your desktop or document and save your main.py there.
4. This particular message will appear if python extension is not setup yet to run in VSCode:



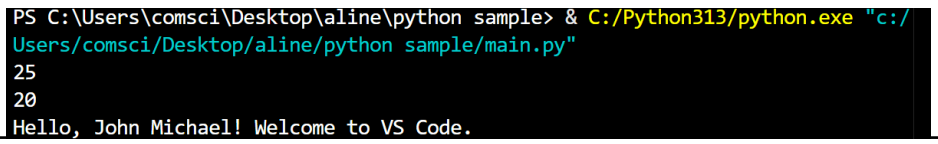

5. Then it will ask you to open a project folder, just select the folder where you had saved your main.py
6. Copy and paste the following code, then press ***Ctrl-S*** to save. The code contains four functions that will add two numbers, multiply two numbers, and print the result of these functions together with a greeting to a student.

```
# math_utils.py

def add_numbers(a, b):
    return a + b

def multiply_numbers(a, b):
    return a * b

def greet_user(name):
    return f"Hello, {name}! Welcome to VS Code."

def main():
    print(add_numbers(5, 3))
    print(multiply_numbers(4, 2))
    print(greet_user("Student"))

if __name__ == "__main__":
    main()
```

7. On the upper right hand corner you will see the play icon  , click it and a terminal window will appear to show you the output, similar to the one shown below:




8. Study the given code, and change it to have an output similar to the one below. **Please replace John Michael with your actual name**.



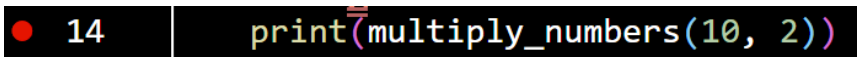
9. Please do the following tasks and then answer the following questions regarding the four basic features of VSCode.

   Submissions of Answers would depend on your teacher, on paper or inside Khub or Google Classroom, so please wait for their instructions. This is an exploratory but graded exercise to let you get to know VSCode.

   a. **Syntax Highlighting**
      ***How does syntax highlighting help you quickly distinguish between functions, variables, and strings in your code?***

   b. **Code Completion**
      **Task:** Type add_ in the main() function.
      ***How does IntelliSense of VSCode reduce typing effort and prevent errors when calling functions?***

   c. **Debugging Tools**
      **Task:** Set a breakpoint on the line **print(multiply_numbers(4, 2))**by clicking on the area before the line number. A red dot will appear before the line number.

      Run the debugger by pressing the ***F5 key.***

      It will stop on the breakpoint and the image below appears beside the filename:

      This will allow you to continue, step-over, step-into, step-out of the function or line, or to stop the debug feature and try to inspect the variable values.

      After trying the debugging tools: ***What information does the debugger provide that helps you understand the program's flow?***

   d. **Error Detections**
      **Task:** Introduce a bug by removing the closing parenthesis from the line print(add_numbers(5, 3)).
      ***How does VS Code indicate errors, and how does this help you fix them before running the program?***

**10. Reflection Questions:**

   a. How do these features compare to using a plain text editor or the IDLE python editor or shell?
   b. Which VSCode Feature will help you catch mistakes and enable you to make the necessary corrections? Why?

**Refer to Annex A: Rubrics for Grading for Activity Part A**
**Total Points: 20points**

## Part B: Setting-up CS3 Code Portfolio

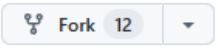
The CS3 Portfolio is a github repository that you will set up and update every time a coding exercise (graded or non-graded) is given to you by your teacher. This will allow you to document your code and could be re-used and become one of your references when you get into CS4 next school year. CS4 is a continuation of CS3 but focuses more on software engineering that applies everything that you will learn this school year.

Each graded exercise will have a grading component regarding your code documentation using your portfolio.

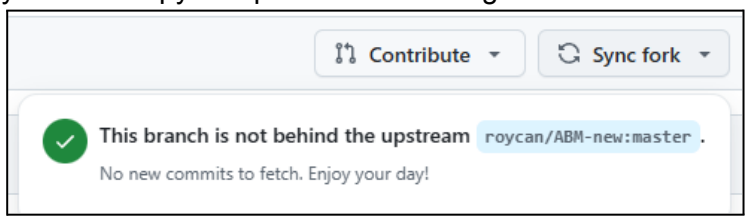
**This activity is a required non-graded exercise.**

**Please follow the instructions:**

1. Login to your github account.
2. Go to a github repository to be provided by your teacher.
3. Make a copy (or fork) the repository by clicking on the github icon.

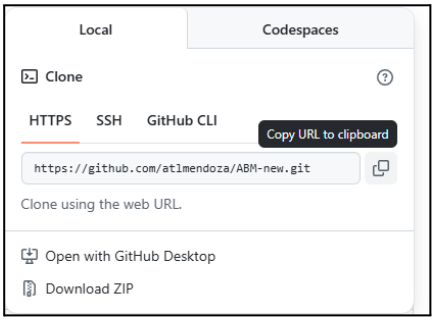
   This could be found on the upper right hand corner of the repository to be copied. The fork icon shows how many times a repository has been copied.
4. The **Create a new repository** of github will appear and fill-out the required information then click on 
   
   .
5. You should now have a copy of the repository under your account. Below the Code green button, you will see the option **Sync fork**, which enables you to get updates on the changes made on the original repository in case your teacher has any updates regarding your portfolio. But you can now update your own copy independent of the original.




**<u>Linking your VSCode to your CS3 Portfolio</u>**

6. Click on the 

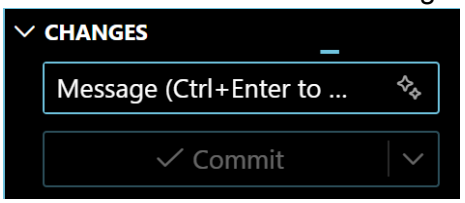
button then copy the .git URL.



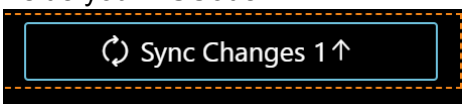

7. Open the VSCode application from your computer. Search for the application. If you do not have a VSCode inside your computer, ***please follow the steps provided in the additional resources at the end of this activity.***
8. Once VSCode is open, press ***Ctrl-Shift-G*** to open source control and click on Clone Repository. **Paste the copied link done in #6 on the space provided by VSCode.**

9. Then, it will ask you where to save it on your local computer. Please create a new folder, using a descriptive name, press enter, then select it.
10. Follow and answer the prompts given by VSCode, then click ***Yes, I trust the authors***
11. At this point, you should see a set of files and folders inside the VSCode Editor.
12. Edit and Personalize README.md using VSCode, by including your name and section plus other information you would like to include on the display. Save by pressing Ctrl-S. **Refer to the additional resources given below regarding how to update a markdown (.md) file.**
13. Update your Github repository with the changes by clicking on this icon on the left side of VSCode 

14. You will see the prompt below. Type in a message to describe the changes to be made on Github before clicking on Commit.




15. If this will show-up, then just click it to synchronize your github to the latest one inside your VSCode.

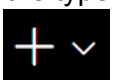



16. Repeat the above process with your computer at home. So that you will be able to update your github wherever you are.

17. **For any problems with the steps, please call the attention of your teacher.**

**ILP Activity : Preparing your work environment**

The following instructions should be **DONE ONLY** if this is the first time connecting your VSCode to your github account. This is assuming that git had been installed in your personal computer and also in the school's laboratory. **(If not, please follow the steps provided in the additional resources at the end of this activity)**

1. Press **Ctrl-` (Control key and the tick mark key)** on your keyboard while you are inside VSCode
2. A terminal window will appear.
3. Change the type of terminal window to **Git Bash** or **Command Prompt** by clicking  on the down arrow, then select from the option given.
4. Type the following, one after the other, after each enter key.
    a. git init
    b. git config --global user.name "github username"
    c. git config --global user.email "your email used in github"

**Additional resources if you have not done any of the following in CS2.**

1. Creating your account in Github account: Create a GitHub Account in 2 Minutes | Beginner Friendly (2026) or https://bit.ly/4kACN5z (duration: 2:48 minutes)

2. Linking your VSCode to your Github Account : How To Connect GitHub To Visual Studio Code 2026 (Step by Step) - YouTube or https://bit.ly/4aiaBAO (duration: 4:52 minutes)

3. Creating your Github Repository How to create your first GitHub repository: A beginner's guide | Tutorial - YouTube or https://bit.ly/4kD1BK5 (duration: 0:55 to 2:55 minutes)

4. To refresh you on how to edit a markdown file please refer to the following two links to websites created by Mr. Roy Canseco of PSHS-MC

    a. Markdown Magic -Your Guide to Easy Formatting or https://bit.ly/4aRRPjV
    b. Markdown for High Schoolers or https://bit.ly/4rPZkxk

5. **Installing VSCode**

    a. How to install Visual Studio Code on Windows 10/11 [ 2025 Update ] VS Code Complete Guide or https://bit.ly/4rQ5k9t (duration: 4:05 minutes) or

    b. How to Install Visual Studio Code on Mac for Beginners | Set up VS Code on Mac OS Step-by-Step Guide or https://bit.ly/46EIklC (duration: 2:57 minutes)

6. **VSCode with Python,** please follow this youtube video: Getting Started with Python in VS Code (Official Video) or https://bit.ly/4kLogEc (from 0:24up to

4:50) . You can continue watching the rest of the video for other tips and tricks using this particular programming language with VSCode.

7. **Downloading and installing Git**

    a. How to Download & Install Git on Windows 10/11 (Latest Version) or https://bit.ly/3Ophwj1 (duration: 2:36 minutes

    b. Install Git on Mac Using Homebrew (Fast & Easy | 2026) or https://bit.ly/4oNAZYW (duration: 2:20 minutes)

## Summary

Selecting and using the right Integrated Development Environment or IDE will help you to work effectively in developing your code and applications. IDEs have common features like syntax highlighting, AI IntelliSense to complete or suggest lines of codes, error detections, and debugger to help you resolve issues with your code.

IDEs -could also integrate with a desktop and cloud application/platform, which is an industry standard for documenting your code or application, which could also be used for version control and collaboration when you do group work.

## Key Takeaways

After setting-up your IDE and connecting it to github, what are your key take-aways regarding this particular part of the CS3 course?

## Resources:


GeeksforGeeks. (2025a, July 18). *Introduction to AIPowered IDEs*. GeeksforGeeks. https://www.geeksforgeeks.org/websites-apps/introduction-to-ai-powered-ides/

GeeksforGeeks. (2025, November 20). *What is an IDE? Integrated Development Environment*. GeeksforGeeks. https://www.geeksforgeeks.org/blogs/what-is-ide/

Gillis, A. S., & Silverthorne, V. (2024, April 29). *integrated development environment (IDE)*. Search Software Quality. https://www.techtarget.com/searchsoftwarequality/definition/integrated-development-environment

GitHub. (2025, November 17). *What is an integrated development environment (IDE)? GitHub.* *https://github.com/resources/articles/what-is-an-ide*

Matt Palmer. (2025, March 27). *What is Vibe Coding? [Video]. YouTube.* https://www.youtube.com/watch?v=5OWurmg41tI

What is an integrated development environment (IDE)?. *(n.d.). ibm.com.* https://www.ibm.com/think/topics/integrated-development-environment

Prepared by:

ALINE TERESA L. MENDOZA
PSHS-MC

Reviewed by:

PABLO M. VILORIA, JR. PhD.
PSHS-CARC

JASMIN C. ESPERANTE
PSHS-NMC

## *Annex A*
## *Rubrics for Grading for Activity Part A*

| Criteria | Excellent (5) | Good (4) | Fair (3) | Needs Improvement (2) | Minimal/None (1) |
|---|---|---|---|---|---|
| **Syntax High-lighting** | Clearly identifies how syntax highlighting distinguishes keywords, variables, and strings, and explains its importance for readability and error prevention. | Identifies syntax highlighting and mentions usefulness, but explanation is limited. | Notices syntax highlighting but gives minimal or unclear explanation of its role. | Mentions syntax highlighting vaguely without connecting to readability or debugging. | Does not recognize or explain syntax highlighting. |
| **Code Comp-letion (Intel-liSense)** | Demonstrates IntelliSense effectively, uses it to complete function calls, and explains how it improves efficiency and reduces errors. | Uses IntelliSense and notes usefulness but explanation lacks depth. | Attempts IntelliSense but shows limited understanding of its benefits. | Mentions IntelliSense vaguely without demonstration. | Does not use or recognize IntelliSense. |
| **Debugging Tools** | Sets breakpoints correctly, runs the debugger, inspects variables, and explains how debugging helps understand program flow. | Sets breakpoints and runs debugger but explanation of debugging benefits is partial. | Attempts to set breakpoints but struggles with debugger usage or explanation. | Mentions debugging vaguely without demonstration. | Does not attempt debugging or shows no understanding of breakpoints. |

| | | | | | |
|---|---|---|---|---|---|
| **Error Detect-ion** | Introduces a bug, observes VS Code's error indicators, and explains how these help fix issues before running the program. | Introduces a bug and notices error indicators but explanation is limited. | Introduces a bug but gives minimal or unclear explanation of error detection. | Mentions error detection vaguely without demonstration. | Does not attempt or recognize error detection |
| **Reflections (as a two point bonus)** | | | | | |